

passgen

Детерминированный генератор паролей

Полная документация проекта для защиты
Подробное описание всех функций программы

1. Идея проекта и решаемая проблема

В современном мире у каждого человека есть множество аккаунтов: электронная почта, социальные сети (ВКонтакте, Telegram), рабочие сервисы (GitHub), образовательные платформы, онлайн-банкинг, интернет-магазины. Для каждого требуется пароль. Использовать один и тот же пароль везде очень опасно: если злоумышленник узнает пароль от одного сервиса (например, из-за утечки базы данных), он получит доступ ко всем аккаунтам. Запоминать 20-30 разных сложных паролей человеческая память не позволяет.

Менеджеры паролей (LastPass, Bitwarden) хранят все пароли в одном зашифрованном месте, но это создает единую точку отказа. Если злоумышленник получит доступ к мастер-паролю от менеджера или если сам менеджер будет взломан, все пароли окажутся скомпрометированы.

passgen предлагает принципиально другой подход — детерминированную генерацию. Программа не хранит пароли, а каждый раз вычисляет их заново с помощью криптографического алгоритма HMAC-SHA256. Пользователю нужно запомнить всего одну фразу (сид). Пароль для любого сервиса восстанавливается по формуле: $\text{пароль} = \text{HMAC-SHA256}(\text{сид}, \text{сервис} + \text{длина} + \text{шаг})$.

Преимущества:

- Пароли не нужно хранить — они всегда восстанавливаются из сида и названия сервиса
- Нечего красть — на компьютере нет базы данных с паролями
- Пароль невозможно потерять — при знании сида он восстанавливается в любой момент
- Для каждого сервиса создается уникальный пароль
- Не требуется интернет
- Программа бесплатна и имеет открытый исходный код

2. Структура программы (index.html)

Вся программа находится в одном файле — index.html. Он состоит из трех частей: HTML (разметка страницы), CSS (стили и оформление) и JavaScript (логика работы). Никаких дополнительных файлов, библиотек или серверов не требуется. Файл открывается в любом современном браузере.

2.1. HTML — разметка страницы

Страница состоит из блоков (карточек), расположенных вертикально сверху вниз:

1. Верхняя панель с заголовком "passgen" и ссылками "Экспорт"/"Импорт".
2. Карточка "Сид" — поле для секретной фразы и место для сообщения об ошибке.
3. Карточка "Сервис" — поле ввода названия сервиса и кнопка "список".
4. Карточка "Параметры" — настройки генерации.
5. Кнопка "Создать".
6. Карточка "Логин и пароль" — результат генерации.
7. Кнопка "Копировать".
8. Модальное окно импорта (скрыто по умолчанию).

2.2. CSS — стили и оформление

Цвета задаются через CSS-переменные в блоке :root. Основные цвета: светло-фиолетовый фон страницы, белые карточки, темный текст, фиолетовый акцентный цвет. Кнопки имеют градиентную заливку, карточки — скругленные углы и тень. Реализованы анимации: плавное появление элементов (fadeUp), пульсация границы при ошибке (glow), раскрытие панелей (slideDown/slideUp), переключение кружочков (checkPop).

2.3. JavaScript — логика программы

На JavaScript написана вся логика программы. Используется Web Crypto API — встроенная в браузер криптографическая библиотека, доступная в Chrome, Firefox, Edge, Safari. Она позволяет вычислять HMAC-SHA256 и SHA-256 прямо на устройстве пользователя без отправки данных в интернет. В коде реализованы: генерация паролей, сохранение и восстановление настроек для каждого сервиса, экспорт и импорт настроек в JSON, обработка всех событий (клики, ввод текста, клавиши).

3. Криптографический алгоритм

3.1. Что такое HMAC-SHA256

HMAC-SHA256 — это международный криптографический стандарт (RFC 2104, FIPS PUB 180-4). Он используется в банковских системах для подтверждения транзакций, в блокчейне биткоина для создания цифровых подписей, в протоколах TLS/SSL для защищенного соединения с сайтами, в защите Wi-Fi (WPA2). Алгоритм берет два входных параметра — секретный ключ и сообщение — и вычисляет 256 бит (32 байта) данных.

У HMAC есть два важнейших свойства. Первое: результат невозможно предсказать без знания ключа. Даже если изменить один символ в ключе или в сообщении, результат изменится полностью. Второе: для одинаковых ключа и сообщения результат всегда одинаковый. Это свойство называется детерминированностью. Благодаря ему можно восстановить пароль в любой момент — достаточно ввести те же самые данные.

3.2. Пошаговая схема генерации пароля

Шаг 1. Пользователь вводит сид (секретную фразу). Программа преобразует эту фразу в набор чисел (байтов) с помощью функции `TextEncoder`. Из этих байтов создается криптографический ключ через функцию `crypto.subtle.importKey`.

Шаг 2. Формируется сообщение для алгоритма. Программа берет название сервиса (например, "google"), добавляет к нему желаемую длину пароля (например, 16) и номер шага. Номер шага начинается с 0 и увеличивается каждый раз, когда программе нужно больше случайных данных. Все это объединяется в одну строку и превращается в байты.

Шаг 3. Вычисляется HMAC-SHA256. Программа вызывает `crypto.subtle.sign("HMAC", key, message)`. Результат — 32 байта криптостойких случайных данных. Эти данные невозможно предсказать без знания ключа (сиды).

Шаг 4. Превращение байтов в символы пароля. Из 32 полученных байтов каждый проверяется: если его значение меньше максимального порога (для равномерного распределения), он превращается в символ из выбранного пользователем алфавита. Если байт не подходит — он пропускается. Если в одном блоке не хватило подходящих байтов, номер шага увеличивается, и вычисляется следующий блок. Процесс повторяется, пока не наберется нужное количество символов.

Шаг 5. Применяются опции инверсии. Если пользователь включил "Первую заглавную" — первый символ пароля делается заглавным. Если включил "Последнюю цифру" — последний символ заменяется на цифру, которая вычисляется отдельным HMAC, чтобы не нарушить детерминированность основного пароля.

3.3. Rejection Sampling (равномерное распределение)

Число 256 (количество возможных значений одного байта) не делится нацело на длину алфавита. Допустим, алфавит состоит из 50 символов. Если бы программа просто брала остаток от деления байта на 50 для всех байтов (от 0 до 255), то символы с номерами 0-5 встречались бы 6 раз, а символы 6-49 — только 5 раз. Это создало бы смещение (bias), которое снижает криптостойкость.

Решение: программа вычисляет $\text{max} = 256 - (256 \% \text{длина_алфавита})$. Байты, значение которых больше или равно max , отбрасываются. Используются только байты меньше max . Для них распределение остатка от деления будет строго равномерным. Пример: алфавит из 50 символов. $\text{max} = 256 - (256 \% 50) = 250$. Байты от 250 до 255 отбрасываются. Каждый символ алфавита встречается ровно 5 раз ($250 / 50 = 5$). Этот метод называется Rejection Sampling (отбраковочная выборка) и является стандартной техникой в криптографии.

3.4. Алфавиты наборов символов

Каждый набор символов содержит строго определенный список. Из некоторых наборов исключены потенциально путающиеся символы.

Заглавные: ABCDEFGHJKLMNPQRSTUVWXYZ (исключены I и O — путаются с 1 и 0).

Строчные: abcdefghikmnpqrstuvwxyz (исключены j, l, o — путаются с цифрами).

Цифры: 0123456789

Спецсимволы: !@#\$%&*+=-.

По умолчанию включены: заглавные, строчные, спецсимволы. Длина 10 символов.

3.5. Генерация логина

Логин генерируется отдельно от пароля, через алгоритм SHA-256 (родственный HMAC, но без ключа). Программа берет сид, добавляет к нему слово "login", название сервиса и значение счетчика. Счетчик увеличивается при каждом вызове функции, поэтому каждый клик по кнопке "Создать логин" дает новый, отличающийся логин.

Из 32 байтов хеша берутся первые 6 и превращаются в символы из специального алфавита: "abcdefghijklmnopqrstuvwxyz23456789" (32 символа). Из алфавита исключены буквы i, l, o (их можно перепутать с цифрами 1 и 0), а также цифры 0 и 1 (их можно перепутать с буквами). В результате получается логин вроде "abc34x", который легко продиктовать по телефону.

4. Интерфейс программы

Подробное описание каждого элемента пользовательского интерфейса.

4.1. Поле "Сид"

Это самое важное поле во всей программе. Сюда пользователь вводит свою секретную фразу — единственное, что нужно запомнить. Без сида невозможно восстановить пароль. Если сид не введен, при попытке генерации поле подсвечивается розовой пульсирующей границей (анимация glow) и появляется сообщение "Введите сид". При начале ввода ошибка исчезает. Нажатие клавиши Enter в этом поле запускает генерацию пароля.

4.2. Поле "Сервис" и список сервисов

В это поле вводится название сайта или сервиса: google, github, vk, yandex и так далее. Справа от поля находится кнопка "список" со стрелкой, которая открывает панель со списком сервисов. В панели отображаются популярные сервисы с цветными кружками-логотипами, а также сервисы, которые пользователь уже использовал ранее.

Популярные сервисы: Google (синий G), GitHub (черный GH), VK (синий V), Яндекс (красный Ya), Mail (синий @), iCloud (темно-синий i), Dropbox (синий D), Telegram (голубой T).

При клике на сервис из списка его название подставляется в поле, а сохраненные настройки (длина, символы, инверсия) автоматически восстанавливаются. Если нужного сервиса нет в списке, можно ввести его название вручную — после генерации пароля он автоматически добавится в список. Правый клик на сохраненном сервисе удаляет его из списка.

4.3. Параметры генерации

Выпадающий список пресетов: "Минимальный (8, буквы+цифры)", "Стандартный (10, все кроме спецсимволов)", "Максимальный (16, все)". Пункт "Свой" означает ручную настройку.

Длина пароля от 1 до 99. Стрелочки для быстрой подстройки без клавиатуры. Четыре кружочка — наборы символов. Фиолетовый = включено. Расширенные настройки: "Первая заглавная" и "Последняя цифра" (скрыты по умолчанию).

4.4. Кнопка "Создать"

Главная кнопка. Запускает полный цикл: проверка сида и чекбоксов, HMAC-SHA256, инверсия, отображение, копирование, добавление в список, сохранение. Фиолетовый градиент, тень. Enter в полях сида/сервиса равносителен нажатию.

4.5. Результат

Логин (только чтение) + "Создать логин". Пароль (только чтение) — моноширинный, жирный, по центру. Индикатор: 3 полоски, цвет от розового до сиреневого, рекомендация.

4.6. Кнопка "Копировать"

Копирует пароль в буфер. Clipboard API, при ошибке exesCommand. "Скопировано!" на 1.5с.

4.7. Экспорт и импорт

Экспорт: JSON со всеми сервисами и настройками.

Импорт: загружает JSON, окно выбора с чекбоксами.

5. Все функции JavaScript (подробное описание)

В этом разделе подробно описывается каждая функция программы. Описания написаны человеческим языком, без использования программного кода, чтобы их можно было устно пересказать на защите проекта.

5.1. Функция `init()` — подготовка программы к работе

Что делает: функция запускается автоматически, когда пользователь открывает страницу программы. Она подготавливает все элементы интерфейса к работе — устанавливает начальные значения, привязывает обработчики событий к полям и кнопкам.

Как работает: сначала `init()` вызывает `loadState()` для очистки всех данных, которые могли остаться от предыдущего использования. Затем она убирает любые ошибки с поля сида — снимает розовую подсветку и очищает текст сообщения об ошибке. После этого она привязывает к полям специальные обработчики событий. Поле сида получает обработчик `input`: при каждом вводе текста ошибка снимается. Поля сида и сервиса получают обработчик `keydown`: при нажатии Enter запускается генерация пароля. Поле сервиса получает обработчик `blur`: при потере фокуса программа запоминает, какой сервис сейчас выбран, и если для него есть сохраненные настройки — применяет их.

Затем `init()` проверяет, есть ли сохраненные настройки для текущего сервиса. Если есть — восстанавливает их (длину, чекбоксы, инверсию). Если нет — устанавливает стандартные настройки: заглавные буквы включены, строчные включены, спецсимволы включены, цифры выключены, длина 10 символов, опции инверсии выключены. В конце функция отрисовывает список сервисов с логотипами.

5.2. Функция loadState() — очистка всех данных

Что делает: полностью очищает все данные программы, чтобы каждый запуск начинался с чистого листа. Это ключевая функция для обеспечения безопасности.

Как работает: функция удаляет все данные, которые могли сохраниться в браузере от предыдущих сессий (localStorage). Обнуляет объект настроек perSvc, который хранит настройки для каждого сервиса. Очищает массив svcs, в котором хранятся названия использованных сервисов. Сбрасывает текущий сервис curSvc на "(empty)". Устанавливает стандартные настройки: заглавные+строчные+специальные символы, без инверсии. Ставит длину пароля 10. Очищает поле ввода сервиса.

Почему это важно: благодаря этой функции, если вы закрыли программу и открыли снова, никаких следов предыдущего использования не останется. Даже названия сервисов, для которых вы генерировали пароли, не сохраняются между сессиями.

5.3. Функция `saveCfg()` — сохранение настроек для сервиса

Что делает: запоминает текущие настройки (какие чекбоксы включены, какая длина, какие опции инверсии) и сохраняет их для текущего сервиса. Благодаря этому при повторном выборе того же сервиса все настройки восстанавливаются автоматически.

Как работает: функция берет текущие значения всех элементов управления: состояние четырех чекбоксов (заглавные, строчные, цифры, спецсимволы) из объекта `window.state`, состояние двух опций инверсии (первая заглавная, последняя цифра) из объекта `window.inv`, и текущую длину пароля из поля ввода. Все это упаковывается в объект и сохраняется в `perSvc` под именем текущего сервиса (`curSvc`).

Важно: настройки хранятся только в оперативной памяти. При закрытии или перезагрузке страницы они безвозвратно исчезают. Это сделано для безопасности.

5.4. Функция `applyCfg()` — восстановление настроек сервиса

Что делает: применяет сохраненные настройки для выбранного сервиса к интерфейсу. Когда пользователь кликает на сервис в списке, программа должна восстановить те настройки, которые использовались для этого сервиса в последний раз.

Как работает: функция принимает объект `cfg` с сохраненными настройками. Она устанавливает длину пароля из `cfg.len`. Восстанавливает состояние всех четырех чекбоксов из `cfg.state`. Включает или выключает опции инверсии из `cfg.inv`. После этого перерисовывает чекбоксы (`renderChk`) и инверсию (`renderInv`), чтобы пользователь видел актуальные настройки.

5.5. Функция `toggleInv()` — переключение расширенных опций

Что делает: переключает состояние опций "Первая заглавная" и "Последняя цифра" при клике пользователя.

Как работает: функция принимает идентификатор опции — "cap" для первой заглавной или "dig" для последней цифры. Она меняет значение в объекте `window.inv` на противоположное: если было 0 (выключено), становится 1 (включено), и наоборот. Затем функция обновляет внешний вид соответствующего кружочка: если опция активна — кружок закрашивается фиолетовым, если нет — остается пустым.

Важно: эта функция НЕ сохраняет настройки. Сохранение произойдет только при нажатии кнопки "Создать". Это предотвращает лишние записи в память.

5.6. Функция `applyPreset()` — применение готового пресета

Что делает: устанавливает готовую комбинацию настроек при выборе пресета из выпадающего списка. Это удобно для пользователей, которые не хотят разбираться в деталях настройки.

Как работает: функция проверяет значение выпадающего списка. Если выбрано "custom" (свой) — функция ничего не делает, пользователь настраивает вручную. Если выбран "Минимальный" — устанавливает длину 8 и включает заглавные, строчные и цифры. Если "Стандартный" — длину 10, заглавные, строчные и цифры. Если "Максимальный" — длину 16 и включает все четыре набора (заглавные, строчные, цифры, спецсимволы). После применения настроек функция перерисовывает чекбоксы и сохраняет настройки для текущего сервиса.

5.7. Функция `renderChk()` — отрисовка чекбоксов символов

Что делает: создает на экране четыре строки с кружочками и названиями наборов символов. Это те самые кружочки, которые пользователь видит в блоке параметров.

Как работает: функция берет массив `CHK`, который содержит четыре элемента: заглавные, строчные, цифры, спецсимволы. Для каждого элемента она создает HTML-строку с кружочком (`span`) и текстом (`span` с названием). Если набор символов активен (`window.state[id]` равен 1), кружочку добавляется CSS-класс `"on"`, который закрашивает его фиолетовым. Каждая строка делается кликабельной: при клике значение переключается (1 становится 0, 0 становится 1), и функция запускается заново для обновления экрана.

5.8. Функция `renderInv()` — отрисовка опций инверсии

Что делает: обновляет внешний вид кружочков для опций "Первая заглавная" и "Последняя цифра".

Как работает: функция берет значения `window.inv.cap` (первая заглавная) и `window.inv.dig` (последняя цифра). Для каждого — если значение равно 1, функция добавляет CSS-класс "on" к соответствующему кружочку; если 0 — убирает этот класс. Простая функция, которая вызывается после изменения настроек инверсии.

5.9. Функция `adj()` — изменение длины пароля

Что делает: обрабатывает клики по стрелкам вверх и вниз рядом с полем длины пароля. Позволяет быстро изменить длину без ввода с клавиатуры.

Как работает: функция принимает параметр `delta`: +1 для увеличения длины на 1, -1 для уменьшения на 1. Она берет текущее значение из поля ввода (`input type="number" с id="length"`), прибавляет к нему `delta`, и записывает результат обратно в поле. При этом функция проверяет границы: если результат меньше 1, устанавливается 1; если больше 99, устанавливается 99.

5.10. Функция toggleSvcPanel() — открытие и закрытие списка сервисов

Что делает: показывает или скрывает панель со списком сервисов при клике на кнопку "список" рядом с полем сервиса.

Как работает: функция находит панель с id="svcPanel" и переключает у нее CSS-класс "open". Когда класс есть — панель видна (max-height: 300px, opacity: 1). Когда класса нет — панель скрыта (max-height: 0, opacity: 0). Анимация раскрытия и сворачивания делается через CSS transition. Стрелка на кнопке поворачивается на 180 градусов, показывая состояние.

5.11. Функция `renderSvcChips()` — сборка списка сервисов

Что делает: собирает все сервисы для отображения в панели списка. Сначала показывает популярные сервисы, затем сохраненные пользователем.

Как работает: функция берет массив `POPULAR` (8 предустановленных сервисов) и проходит по нему. Каждый популярный сервис добавляется в список, если его еще нет в массиве `svcs` (сохраненные сервисы). Затем функция проходит по массиву `svcs` и добавляет все сервисы, которые пользователь уже использовал. Для каждого сервиса вызывается функция `chip()`, которая создает готовый элемент.

5.12. Функция `chip()` — создание кнопки сервиса с логотипом

Что делает: создает один элемент (кнопку) для сервиса с логотипом и названием.

Как работает: функция принимает три параметра. `item` — может быть объектом (для популярных сервисов с цветом и буквой) или строкой (для пользовательских сервисов). `hasLogo` — флаг, показывать ли цветной логотип. `deletable` — флаг, можно ли удалять сервис правым кликом.

Если `hasLogo true` и `item` — объект, функция создает цветной кружок: `div` размером 20x20 пикселей, скругленный (`border-radius: 50%`), с цветом фона из `item.c`, текстом из `item.l` (буква-логотип), белым цветом текста, жирным шрифтом 10px. Если это пользовательский сервис, создается серый кружок с вопросительным знаком.

При клике на кнопку: поле сервиса заполняется названием, панель списка закрывается, применяются сохраненные настройки. Для `deletable` сервисов при правом клике вызывается контекстное меню: сервис удаляется из массива `svcs`, список перерисовывается.

5.13. Функция `gen()` — генерация пароля (главная функция)

Что делает: самая важная функция всей программы. Запускается при нажатии кнопки "Создать" или при нажатии Enter в полях сида или сервиса. Выполняет полный цикл генерации пароля.

Первый шаг — проверка сида. Функция берет значение из поля с `id="seed"`, обрезает пробелы (`trim`). Если строка пустая — добавляет CSS-класс `"error"` на поле (розовая подсветка), показывает текст "Введите сид" в элементе `seedErr`, и прекращает выполнение (`return`).

Второй шаг — проверка наборов символов. Функция считает, сколько наборов включено (`filter(Boolean).length`). Если ни один не включен — показывает сообщение об ошибке и прекращает выполнение.

Третий шаг — составление алфавита. Функция проходит по массиву СНК, для каждого набора, который включен, берет строку символов (третий элемент массива) и добавляет в общий алфавит. Например, если включены заглавные и цифры, алфавит будет `"ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"`.

Четвертый шаг — вычисление пароля. Функция вызывает `hmacGen(seed, svc, len, alpha)` — асинхронную функцию, которая вычисляет HMAC-SHA256. Используется `await` для ожидания результата.

Пятый шаг — инверсия. Если `window.inv.cap` равно 1, функция берет первый символ пароля и делает его заглавным (`toUpperCase`). Если `window.inv.dig` равно 1, функция вычисляет дополнительный HMAC для последней цифры: импортирует ключ из `seed + "_lastdig"`, подписывает сообщение `svc + len`, берет первый байт результата, вычисляет цифру = байт % 10, и заменяет последний символ пароля на эту цифру.

Шестой шаг — отображение. Функция записывает готовый пароль в поле `pwd`.

Седьмой шаг — обновление индикатора надежности. Вызывается `updateStrength(len, sets)`.

Восьмой шаг — копирование в буфер обмена. Функция пытается вызвать `navigator.clipboard.writeText(finalPwd)`. В случае успеха показывает сообщение "Скопировано!" на 2 секунды. Ошибка игнорируется.

Девятый шаг — добавление сервиса в список. Если сервиса нет в массиве `svcs` и он не `"(empty)"`, функция добавляет его, сортирует массив и перерисовывает список сервисов.

Десятый шаг — сохранение. Функция устанавливает `curSvc = svc` и вызывает `saveCfg()` для сохранения настроек.

5.14. Функция hmacGen() — криптографическое ядро программы

Что делает: выполняет actual криптографические вычисления — генерацию пароля с помощью HMAC-SHA256. Это асинхронная функция (async), что означает, что она не блокирует работу браузера во время вычислений. Даже при генерации длинных паролей (99 символов) интерфейс остается отзывчивым.

Функция принимает четыре параметра: seed (сид — строка), svc (сервис — строка), len (длина — число), alpha (алфавит — строка).

Первый шаг — подготовка ключа. seed преобразуется в массив байтов через `TextEncoder().encode()`. Из этих байтов создается криптографический ключ: `crypto.subtle.importKey("raw", keyBytes, {name: "HMAC", hash: "SHA-256"}, false, ["sign"])`. Параметр `false` означает, что ключ нельзя экспортировать. `"sign"` — ключ только для подписи (вычисления HMAC).

Второй шаг — базовая часть сообщения. svc и len объединяются в строку (например, `"google16"`) и преобразуются в байты через `TextEncoder().encode()`.

Третий шаг — вычисление max для rejection sampling. $max = 256 - (256 \% alpha.length)$.

Четвертый шаг — цикл генерации. Пока длина пароля (`pwd.length`) меньше требуемой (`len`), выполняется: создается новое сообщение размером `msgBase.length + 1`, копируется `msgBase`, в последний байт записывается значение счетчика `ct` (начинается с 0). Вычисляется HMAC: `crypto.subtle.sign("HMAC", key, message)`. Результат — `ArrayBuffer`, который превращается в `Uint8Array` (32 байта). Для каждого байта проверяется: если байт $< max$, то вычисляется символ = `alpha[байт \% alpha.length]` и добавляется к паролю. Если длина пароля достигла `len` — цикл прерывается. Счетчик `ct` увеличивается на 1.

Пятый шаг — возврат готового пароля.

5.15. Функция `genLogin()` — генерация логина

Что делает: создает короткий (6 символов) логин, который легко запомнить и продиктовать по телефону. Нужен для сервисов, которые требуют не только пароль, но и имя пользователя.

Как работает: функция сначала проверяет, что сид введен (иначе — ошибка) и что сервис указан (иначе — выход без ошибки). Затем увеличивает значение `loginCounter` (счетчик логинов) на 1. Формирует строку из сида, слова "login", названия сервиса и значения счетчика. Преобразует эту строку в байты и вычисляет SHA-256 через `crypto.subtle.digest("SHA-256", bytes)`. Из 32 полученных байтов берет первые 6. Каждый байт превращается в символ из специального алфавита: "abcdefghijklmnopqrstuvwxyz23456789". Результат записывается в поле `login`.

Благодаря счетчику, который увеличивается при каждом вызове, каждый клик по кнопке дает новый логин. Но при одинаковых сиде+сервисе+счетчике логин будет одинаковым (детерминированность).

5.16. Функция `updateStrength()` — индикатор надежности пароля

Что делает: оценивает качество сгенерированного пароля по двум параметрам — длине и количеству выбранных наборов символов — и отображает результат в виде цветных полосок и текстовой рекомендации.

Как работает: функция принимает два параметра: `len` (длина пароля) и `sets` (количество выбранных наборов символов). Она сравнивает их с пороговыми значениями. Если `len >= 14` и `sets >= 4` — уровень "strong" (отличный), отображается 3 полоски сиреневого цвета и текст "Отличный пароль". Если `len >= 10` и `sets >= 3` — уровень "medium" (средний), 2 полоски малинового цвета, текст "Хороший, можно длиннее". В остальных случаях — уровень "weak" (слабый), 1 полоска розового цвета, текст "Добавьте символов или увеличьте длину".

Функция создает полоски динамически: добавляет `div`-элементы в контейнер `strBars`. Каждая полоска получает соответствующий CSS-класс для цвета.

5.17. Функция `сру()` — копирование пароля в буфер обмена

Что делает: копирует текст сгенерированного пароля в буфер обмена, чтобы пользователь мог вставить его на целевом сайте.

Как работает: функция берет значение из поля `pwd`. Если поле пустое — ничего не делает. Пытается использовать современный API буфера обмена: `navigator.clipboard.writeText(p)`. Если этот метод недоступен (старый браузер, не HTTPS, `file://` протокол), функция использует запасной способ: выделяет текст в поле (`select()`) и вызывает `document.execCommand("copy")`. После успешного копирования показывает сообщение "Скопировано!" на 1.5 секунды.

5.18. Функция `exportData()` — экспорт настроек в JSON

Что делает: создает и скачивает файл в формате JSON, содержащий все сервисы и их настройки. Это позволяет перенести настройки на другой компьютер или сделать резервную копию.

Как работает: функция создает пустой объект `clean`. Проходит по всем сервисам из массива `svcs` и по сервису `"(empty)"`. Для каждого, если в `perSvc` есть сохраненные настройки, добавляет их в `clean`. Формирует объект `data = {svcs: svcs, perSvc: clean}`. Преобразует его в JSON с форматированием: `JSON.stringify(data, null, 2)` — отступы в 2 пробела для читаемости. Создает Blob с типом `application/json`. Создает виртуальную ссылку (элемент `a`), устанавливает `href = URL.createObjectURL(blob)` и `download = "passgen_export.json"`. Программно кликает по ссылке — начинается скачивание. После скачивания ссылка удаляется.

5.19. Функция `importData()` — импорт настроек из JSON

Что делает: загружает ранее экспортированный JSON-файл с настройками и готовит данные к импорту.

Как работает: функция получает событие `event` от `input type="file"`. Берет первый выбранный файл (`event.target.files[0]`). Если файл не выбран — выход. Создает `FileReader`. В обработчике `onload`: парсит содержимое файла как JSON. Проверяет, что в данных есть поля `perSvc` или `svcs`. Если данных нет — показывает сообщение "Файл не содержит данных `passgen`". Если данные корректны — сохраняет их во временную переменную `importDataCache` и вызывает `showImportModal(d)` для показа окна выбора сервисов. Очищает значение `input file`, чтобы можно было повторно выбрать тот же файл.

5.20. Функция `showImportModal()` — окно выбора сервисов для импорта

Что делает: создает и показывает модальное окно со списком всех сервисов из импортируемого файла, чтобы пользователь мог выбрать, какие импортировать.

Как работает: функция берет данные из `importDataCache`. Получает список сервисов: `Object.keys(d.perSvc || {}).sort()`. Если список пуст — показывает сообщение "Нет сервисов для импорта" и выходит. Для каждого сервиса из списка создает строку (label) с чекбоксом (`input type="checkbox", checked=true`) и названием сервиса. Добавляет все строки в контейнер `modallist`. Показывает модальное окно: `document.getElementById("importModal").classList.add("open")`.

5.21. Функция `confirmImport()` — подтверждение импорта

Что делает: завершает импорт — записывает выбранные пользователем сервисы и их настройки в программу.

Как работает: функция берет `importDataCache` (данные из файла). Собирает все отмеченные чекбоксы из `modallist`. Если ни один не отмечен — показывает сообщение "Выберите хотя бы один сервис". Для каждого отмеченного чекбокса: берет название сервиса из `dataset.svc`, копирует настройки из `d.perSvc[svc]` в `perSvc[svc]`, добавляет сервис в массив `svcs` (если его там еще нет). Сортирует `svcs`. Перерисовывает чипы сервисов. Если настройки загружены для текущего сервиса — применяет их. Закрывает модальное окно. Показывает сообщение "Импорт завершен!" на 3 секунды.

5.22. Функция `closeImportModal()` — закрытие окна импорта

Что делает: закрывает модальное окно импорта, если пользователь передумал импортировать, или после успешного завершения импорта.

Как работает: функция убирает CSS-класс "open" с модального окна (`document.getElementById("importModal").classList.remove("open")`) и очищает временные данные (`importDataCache = null`).

5.23. Функция `toggleAdvanced()` — открытие расширенных настроек

Что делает: показывает или скрывает панель с опциями "Первая заглавная" и "Последняя цифра". Панель скрыта по умолчанию, чтобы не занимать место на экране.

Как работает: функция находит панель `advPanel` и проверяет ее текущее состояние. Если панель скрыта (`max-height: 0`), функция устанавливает `max-height = 120px` и `opacity = 1`. Если панель открыта — сбрасывает `max-height = 0` и `opacity = 0`. Переход анимируется через `CSS transition`. Стрелка рядом с текстом поворачивается на 90 градусов при открытии.

5.24. Функция `getCfg()` — получение настроек сервиса

Что делает: вспомогательная функция, которая возвращает настройки для запрошенного сервиса. Если настроек нет — создает стандартные.

Как работает: функция принимает название сервиса. Проверяет, есть ли запись в `perSvc[svc]`. Если нет — создает новую со стандартными значениями: `state: {capitals:1, lowercase:1, digits:0, symbols:1}, inv: {cap:0, dig:0}, len:10`. Возвращает объект настроек.

5.25. Функция onSvcChange() — обработчик смены сервиса

Что делает: вызывается при изменении поля сервиса. Обновляет информацию о текущем сервисе и применяет соответствующие настройки.

Как работает: функция берет значение поля сервиса, обрезает пробелы. Если поле пустое — использует "(empty)". Устанавливает curSvc = svc. Проверяет, есть ли настройки для этого сервиса в perSvc. Если есть — применяет их через applyCfg().

5.26. Функция `saveState()` — заглушка для безопасности

Что делает: в текущей версии программы эта функция пуста. Ранее она сохраняла данные в `localStorage` браузера, но от этого отказались в пользу сессионного режима (данные только в оперативной памяти).

Функция оставлена в коде для обратной совместимости — на случай, если в будущем понадобится снова куда-то сохранять данные. В текущей реализации она просто ничего не делает.

6. Обработка ошибок

В программе предусмотрена обработка следующих ошибочных ситуаций:

1. Пустой сид. Если пользователь пытается сгенерировать пароль без ввода сида, поле подсвечивается розовой пульсирующей границей и появляется сообщение "Введите сид". При начале ввода ошибка автоматически снимается.
2. Не выбран ни один набор символов. Показывается сообщение "Выберите хотя бы один тип символов". Генерация не происходит.
3. Длина пароля выходит за границы (меньше 1 или больше 99). Функция `adj()` принудительно устанавливает граничное значение.
4. Неверный формат импортируемого файла. Если выбранный файл не является корректным JSON или не содержит данных `passgen`, показывается сообщение об ошибке.
5. В импортируемом файле нет ни одного сервиса. Показывается сообщение "Нет сервисов для импорта".
6. Clipboard API недоступен. Функция `cpy()` использует запасной метод `execCommand("copy")`.

7. Экспорт и импорт (формат JSON)

Пример структуры экспортируемого/импортируемого файла:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

`svcs` — массив строк с названиями сервисов.

`perSvc` — объект, где ключ — название сервиса, значение — объект с полями:

`state` — флаги `capitals`, `lowercase`, `digits`, `symbols` (1 — включено, 0 — выключено)

`inv` — флаги `cap` (первая заглавная), `dig` (последняя цифра)

`len` — длина пароля (число от 1 до 99)

8. Безопасность

1. Локальность. Все вычисления происходят на локальном компьютере в браузере. Данные не отправляются в интернет и не могут быть перехвачены при передаче.

2. Сессионный режим. Настройки хранятся только в оперативной памяти. При перезагрузке страницы все данные безвозвратно удаляются.
3. Стандартная криптография. HMAC-SHA256 — международный стандарт (RFC 2104), используется в банковских системах, блокчейне, TLS/SSL.
4. Rejection Sampling. Гарантирует равномерное распределение символов в пароле без смещения вероятности (modulo bias).
5. Необратимость. Зная пароль, невозможно восстановить сид из-за свойств хеш-функции (одностороннее преобразование).
6. Разные пароли для разных сервисов. Название сервиса является частью входных данных HMAC, поэтому пароли для разных сервисов совершенно различны.

Ограничения

Программа не защищает от кейлоггеров (как и любой генератор паролей на компьютере). Не хранит пароли (это особенность, а не ограничение). Не требует мастер-пароля — сид и есть мастер-пароль. Не создает резервных копий — сид нужно хранить отдельно.

9. Запуск программы

Программа представляет собой один файл index.html. Способы запуска:

1. Дважды кликнуть по файлу index.html — он откроется в браузере по умолчанию.
2. Открыть браузер, нажать Ctrl+O, выбрать index.html.
3. Если необходимо, чтобы криптография работала через HTTP (не file://), запустить простой HTTP-сервер:
python3 -m http.server
затем открыть http://localhost:8000

Системные требования:

- Любой современный браузер (Chrome, Firefox, Edge, Safari, Opera)
- Web Crypto API (встроен во все современные браузеры)
- Интернет не требуется
- Python не требуется (опционально для HTTP-сервера)